Improvements in FPGA Technology Mapping

Satrajit Chatterjee, Alan Mishchenko
and Robert Brayton
U. C. Berkeley

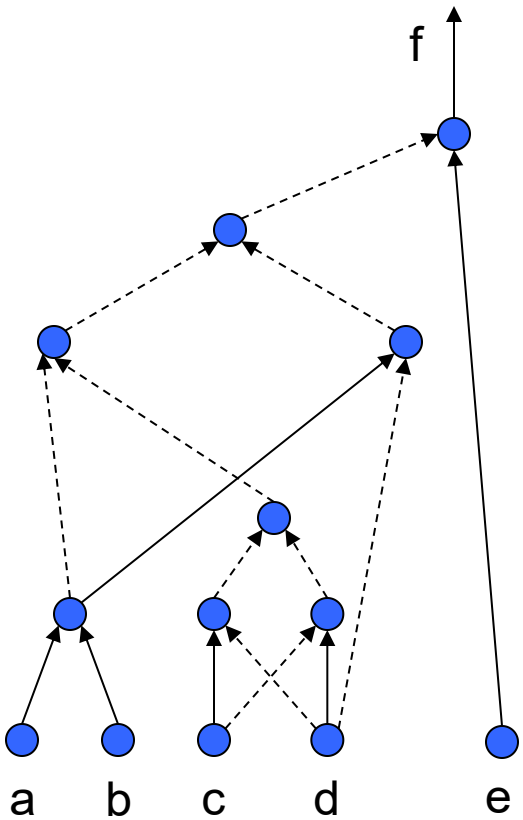
Outline

1. Review of Technology Mapping
2. More Efficient Cut Computation
3. Lossless Synthesis
4. Area Recovery

Technology Mapping

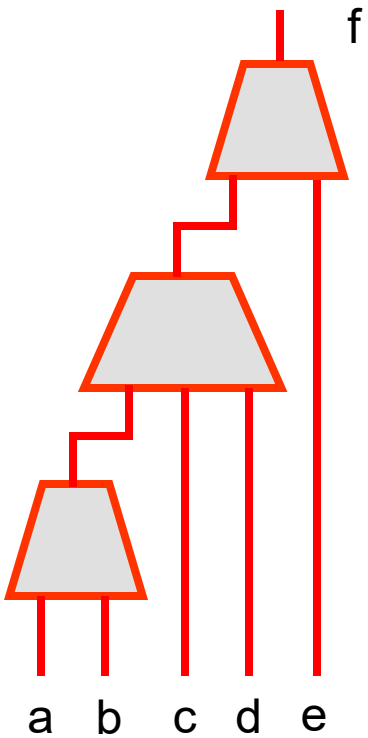
Input: A Boolean network

Output: A netlist of *k*-LUTs implementing the Boolean network optimizing some cost function



The subject graph

Technology Mapping



The mapped netlist

Basic Mapping Algorithm

Cut-based mapping using dynamic programming on a DAG for *delay optimality*

Input: And-Inverter Graph

1. Compute *k*-feasible cuts for each node
2. Compute best arrival time at each node
   - In topological order (from PI to PO)
   - Assuming that each cut maps to a *k*-LUT
   - Assuming that each *k*-LUT has unit delay
3. Chose the best cover
   - In reverse topological order (from PO to PI)

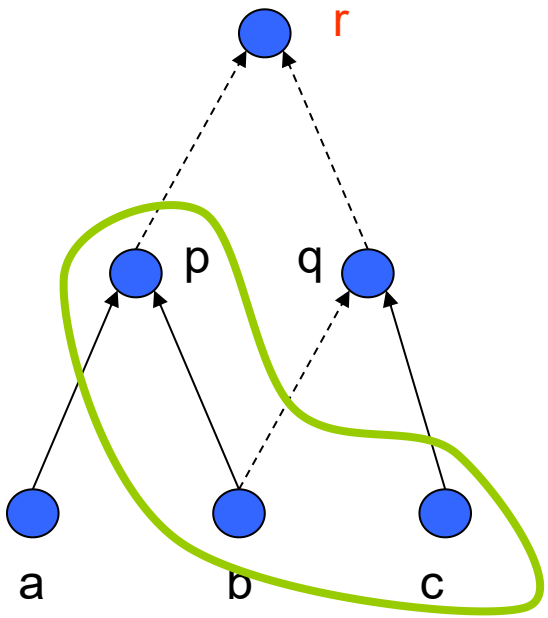
Output: Mapped Netlist

k-feasible Cuts

(Rough definitions)

A *cut* of a node *n* is a set of nodes in transitive fan-in such that assigning values to those nodes fixes the value of *n.*

A k-feasible cut means the size of the cut must be *k* or less.

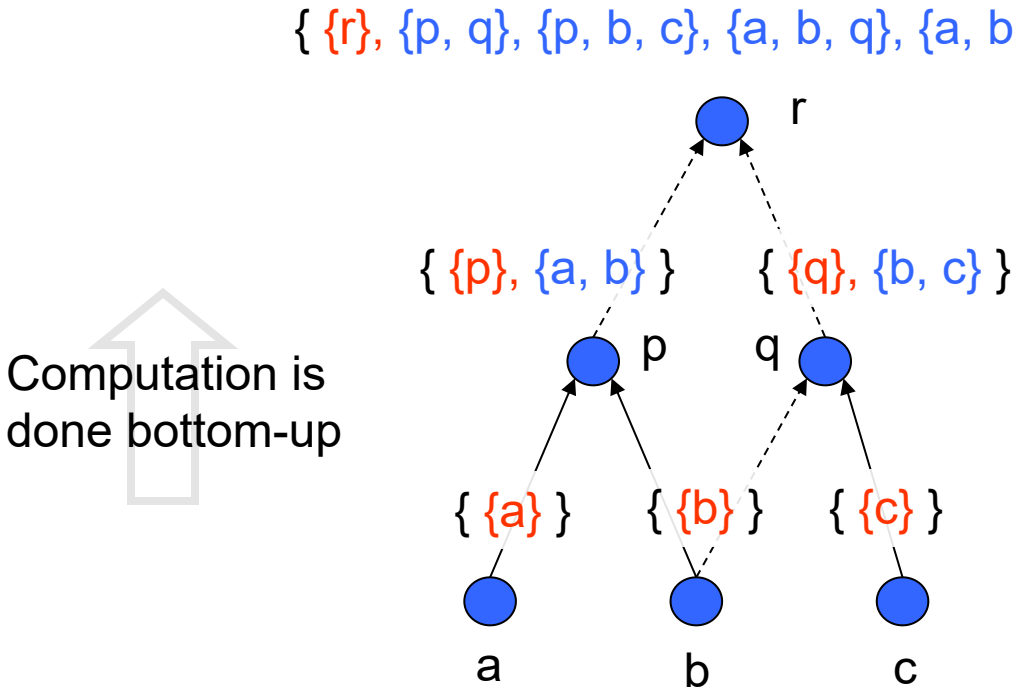


The set {p, b, c} is a 3-feasible cut of node r. (It is also a 5-feasible cut.)

k-feasible cuts are important in FPGA mapping since the logic between a node and the nodes in its cut can be replaced by a *k*-LUT.

k-feasible Cut Computation

The set of cuts of a node is a 'cross product' of the sets of cuts of its children



Any cut that is of size greater than *k* is discarded

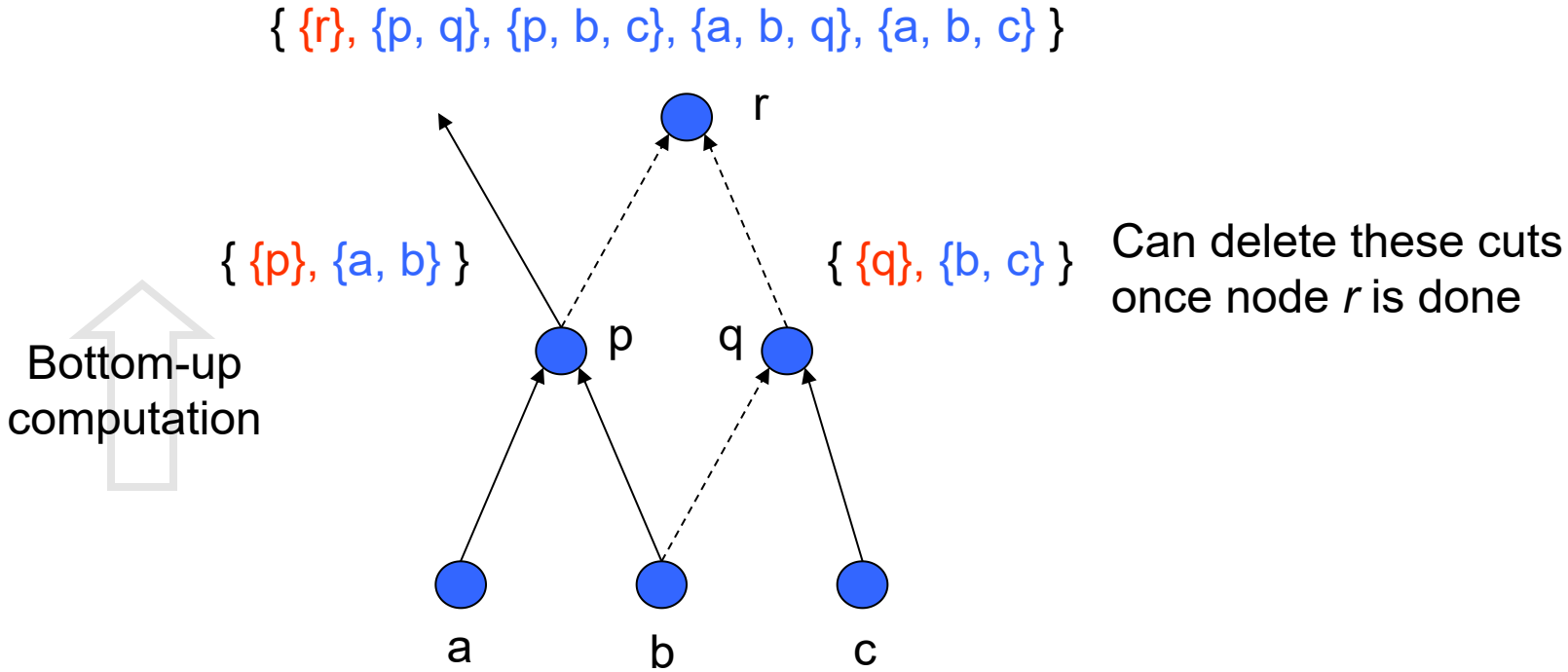
(Pan '98, Cong '99)

Outline

1. Review of Technology Mapping
2. More Efficient Cut Computation
    1. Cut Dropping
    2. Cut Dominance
3. Lossless Synthesis
4. Area Recovery

Cut Dropping

During bottom up computation of cuts, the set of cuts of a node can be freed once all its fan-outs have been processed

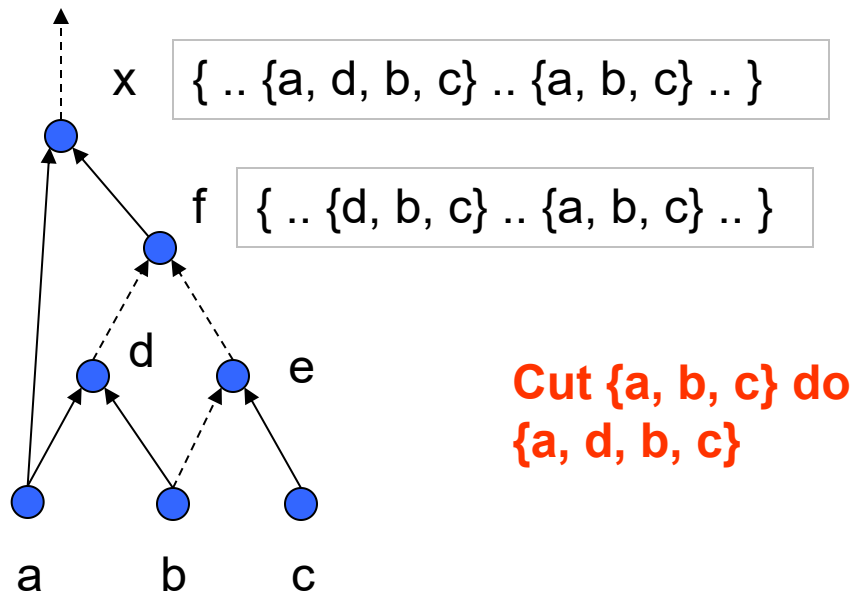


- Once the cuts of node *r* are computed, the cuts of *q* are no longer needed
- But cannot discard the cuts of node *p* since not all fan-outs of p have been processed
- Dramatically reduces peak memory consumption on large designs

Cuts Behaving Badly

Bottom-up cut computation in the presence of re-convergence might produce *dominated* cuts

x = ~a + a.b + ~b.c



Cut {a, b, c} dominates cut {a, d, b, c}

- The “good” cut {a, b, c} is there: so not a quality issue
- But the “bad” cut {a, d, b, c} may be propagated further: so a run-time issue
- Want to discard dominated cuts ***quickly***

Signature-based Dominance

Problem: Given two cuts how to quickly determine whether one is a subset of another

Define *signature* of a cut:

$$\text{sig}(c) = \sum_{n \in c} 2^{ID(n) \bmod 32}$$

(Σ means bit-wise OR)

where ID(n) is the integer id of node *n*

Observation: If cut c_1 dominates cut c_2 then

$$\text{sig}(c_1) \text{ OR } \text{sig}(c_2) = \text{sig}(c_2)$$

Cheap test for the common case that a cut does not dominate another. Only if this fails is an actual comparison made.

Example

- Let the node id's be a = 1, b = 2, c = 3, d = 4
- Let c_1 = {a, b, c} and c_2 = {a, d, b, c}
- sig (c_1) = 2^1 OR 2^2 OR 2^3

  = 0001 OR 0010 OR 0100

  = 0111
- sig (c_2) = 2^1 OR 2^4 OR 2^2 OR 2^3

  = 0001 OR 1000 OR 0010 OR 0100

  = 1111
- As sig (c_1) OR sig (c_2) $\neq$ sig (c_1), c_2 does **not** dominate c_1
- But sig (c_1) OR sig (c_2) = sig (c_2), so c_1 **may** dominate c_2

Other Uses of Signatures

- Signatures can be used as quick negative tests for equality of cuts and for *k*-feasibility

Run-time of *k*-feasible cut computation

		$K = 4$		$K = 5$		$K = 6$		$K = 7$		$K = 8$		
Name	N	C/N	T, s	C/N	T, s	C/N	T, s	C/N	T, s	C/N	T, s	L/N, %
alu4	2642	6.7	0.00	12.3	0.01	23.1	0.04	45.5	0.18	94.7	1.02	0.00
apex2	2940	7.2	0.01	14.2	0.02	29.2	0.07	62.6	0.32	139.7	1.90	0.00
apex4	2017	8.5	0.00	19.5	0.03	47.0	0.10	116.3	0.62	293.5	4.49	0.10
bigkey	3080	6.6	0.01	12.1	0.02	24.2	0.05	50.1	0.20	99.7	0.84	0.00
clma	11869	8.1	0.04	18.2	0.11	44.4	0.51	114.9	3.01	306.3	20.99	1.64
des	3020	8.0	0.01	17.0	0.03	38.7	0.12	92.0	0.69	218.0	4.80	4.37
diffeq	2566	6.5	0.01	12.3	0.01	26.6	0.07	65.0	0.50	155.9	2.80	3.66
dsip	2521	6.2	0.01	10.7	0.01	20.7	0.03	42.0	0.10	86.7	0.44	0.00
elliptic	5502	6.4	0.01	10.6	0.03	18.5	0.07	36.9	0.33	83.4	2.12	0.20
ex1010	7652	9.2	0.02	23.3	0.11	61.8	0.61	165.8	4.01	438.2	30.43	1.99
ex5p	1719	9.4	0.01	24.1	0.02	66.2	0.17	188.2	1.30	514.8	10.50	14.14
frisc	5905	7.1	0.01	14.4	0.04	32.3	0.16	79.8	0.88	209.0	6.30	1.24
misex3	2441	7.7	0.01	15.7	0.02	33.3	0.08	73.7	0.38	170.7	2.48	0.00
pdc	7527	9.4	0.03	24.8	0.12	67.4	0.68	183.7	4.41	489.4	31.71	4.40
s298	2514	7.9	0.00	17.5	0.02	44.0	0.13	121.9	0.94	346.5	7.10	7.56
s38417	12867	6.6	0.03	13.5	0.10	32.0	0.46	83.1	3.24	225.9	23.72	3.38
s38584	11074	6.1	0.03	11.4	0.06	22.4	0.20	46.7	0.98	101.5	5.81	0.86
seq	2761	7.5	0.00	15.2	0.02	31.7	0.08	68.6	0.37	153.3	2.25	0.04
spla	6556	9.6	0.03	25.8	0.11	73.9	0.69	215.5	4.98	561.4	31.14	13.83
tseng	1920	6.5	0.01	11.8	0.01	23.5	0.04	50.6	0.21	112.7	1.32	1.35
Average	**4954.6**	**7.56**	**0.01**	**16.22**	**0.05**	**38.05**	**0.22**	**95.15**	**1.38**	**240.0**	**9.61**	**2.94**

Peak Memory in Mb with Cut Dropping

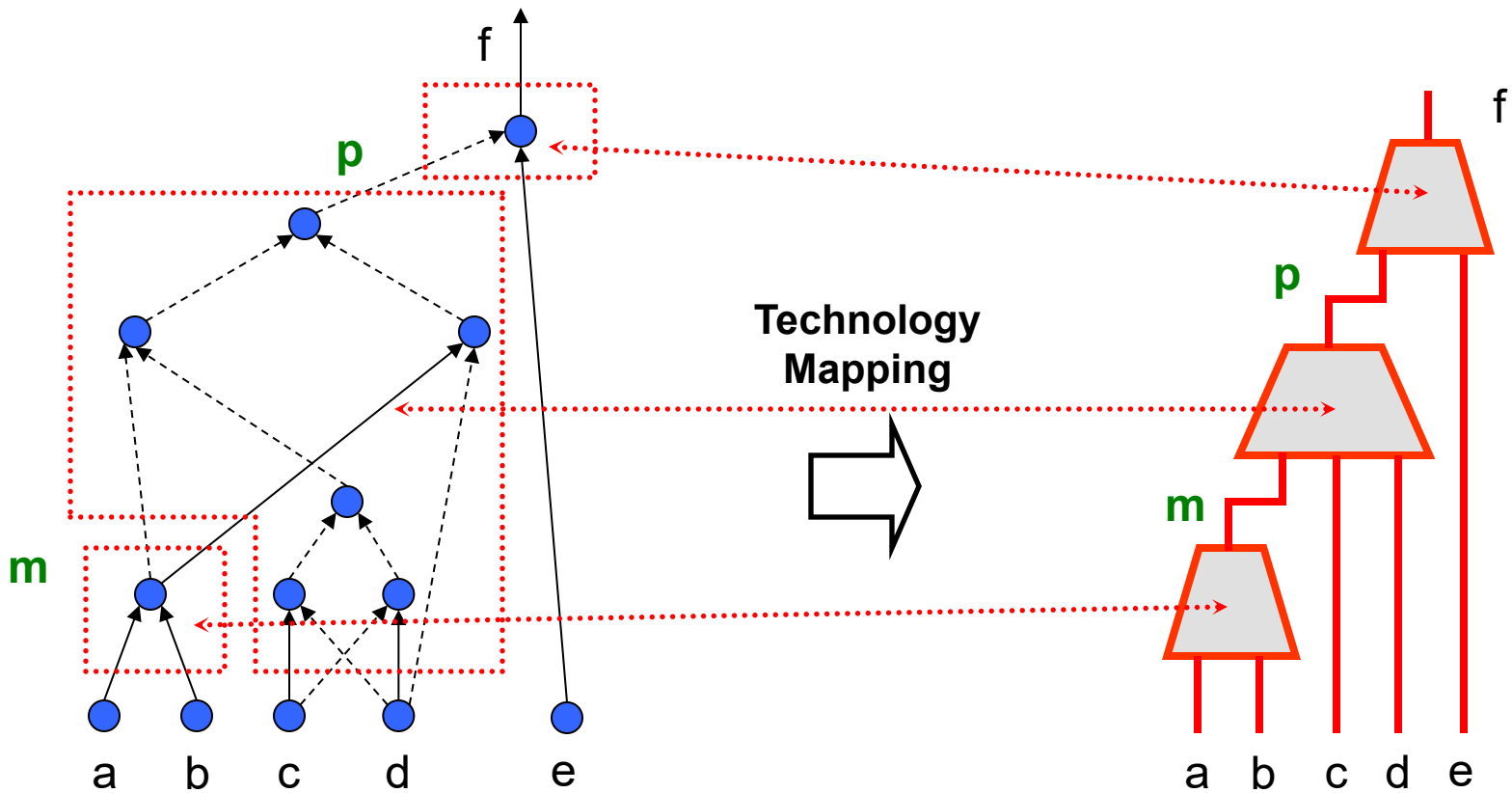
	$K = 4$		$K = 5$		$K = 6$		$K = 7$		$K = 8$	
Name	Total	Drop	Total	Drop	Total	Drop	Total	Drop	Total	Drop
clma	2.56	0.10	6.60	0.22	18.09	0.54	52.03	1.47	152.55	4.07
ex1010	1.87	0.37	5.45	0.97	16.25	2.27	48.40	4.68	140.70	8.38
pdc	1.90	0.27	5.69	0.75	17.42	2.00	52.75	4.98	154.56	11.83
s38417	2.28	0.15	5.28	0.37	14.12	1.10	40.80	3.55	121.98	10.25
s38584.1	1.80	0.11	3.86	0.20	8.52	0.40	19.72	0.86	47.15	1.94
spla	1.68	0.21	5.15	0.59	16.63	1.65	53.88	4.34	154.44	10.04
Ratio	**1.00**	**0.11**	**1.00**	**0.10**	**1.00**	**0.08**	**1.00**	**0.07**	**1.00**	**0.06**

Outline

1. Review of Technology Mapping
2. More Efficient Cut Computation
3. Lossless Synthesis
4. Area Recovery

Structural Bias

The mapped netlist very closely resembles the subject graph

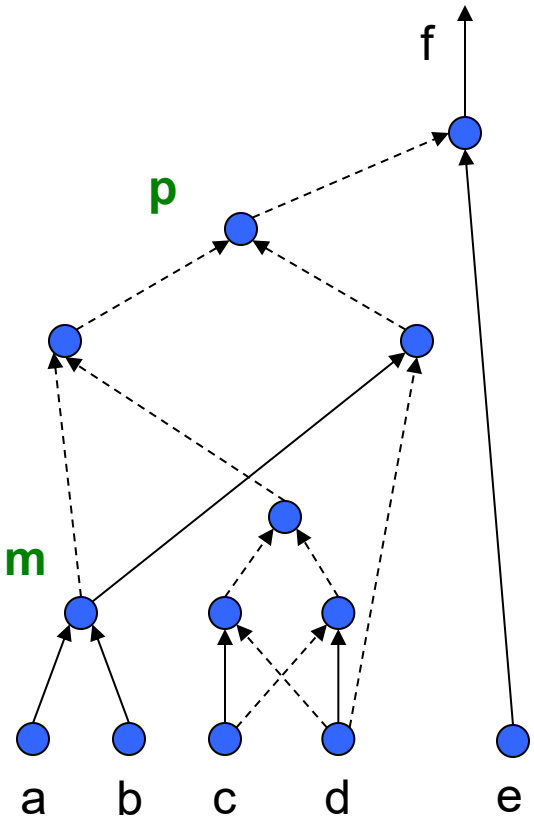


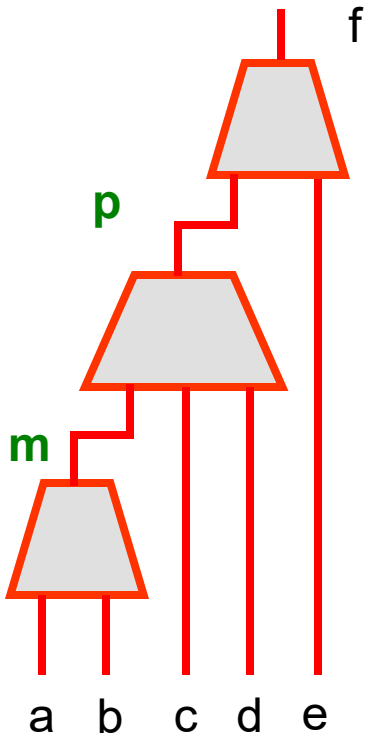
Every input of every LUT in the mapped netlist must be present in the subject graph ..

.. otherwise technology mapping will not find the match

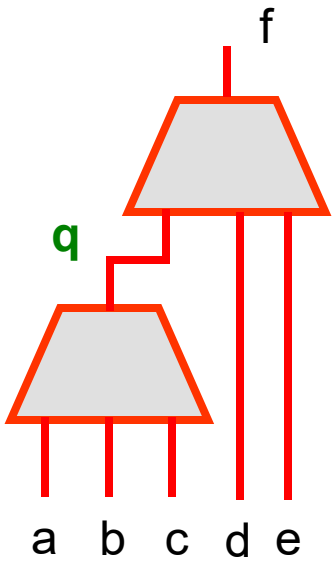
The Problem of Structural Bias

A better match may not be found





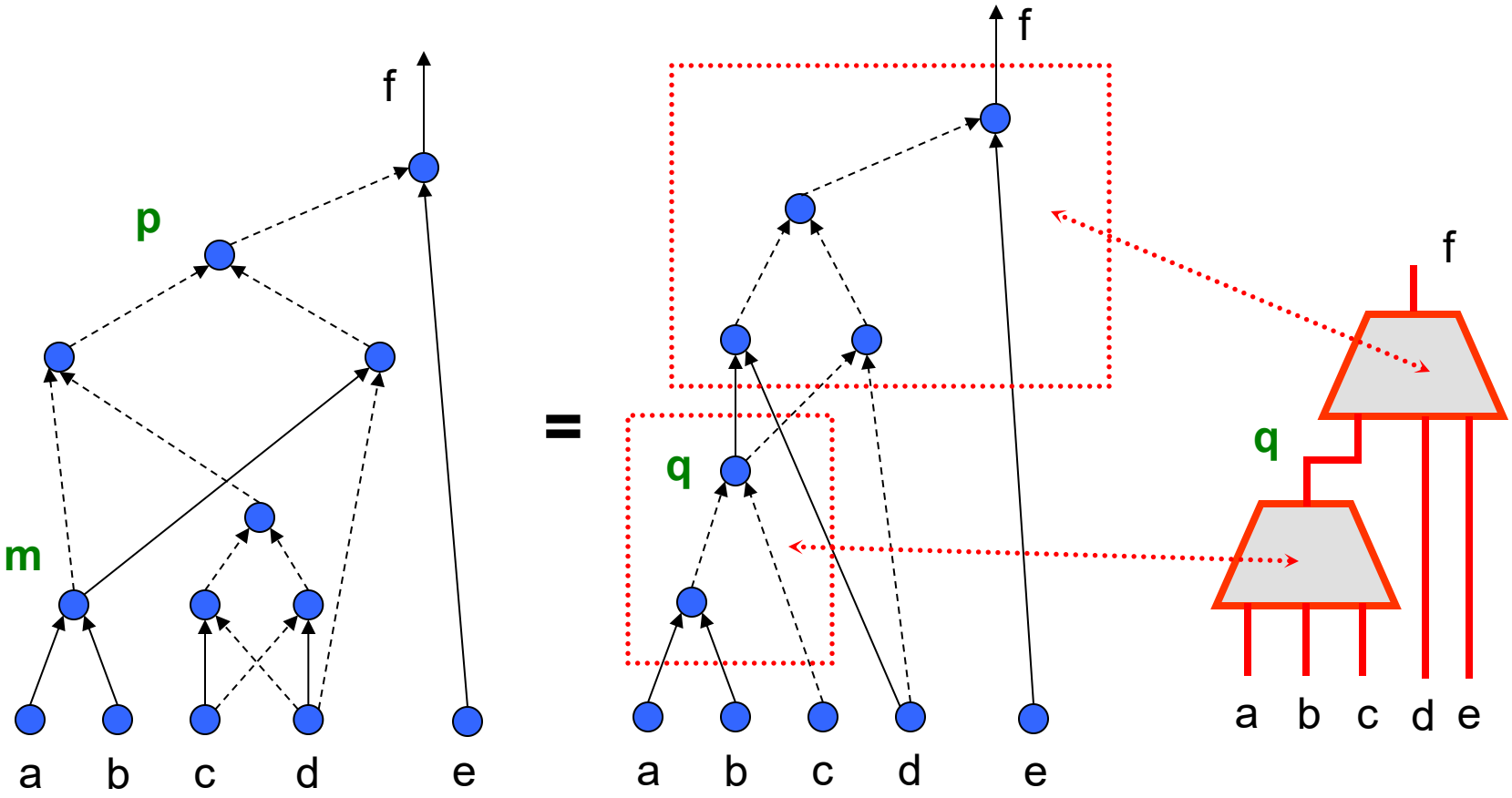
This match is not found



Since the point **q** is **not** present in the subject graph, the match on the extreme right will **not** be found

The Problem of Structural Bias

The match would be found with a different subject graph



Traditional Synthesis

Only the network at the end of technology independent synthesis is used for mapping

Technology-independent synthesis

sweep

eliminate
resub
simplify

fx
resub
sweep

eliminate
sweep
full simplify

Boolean Network

No guarantee of optimality since each synthesis step is heuristic.

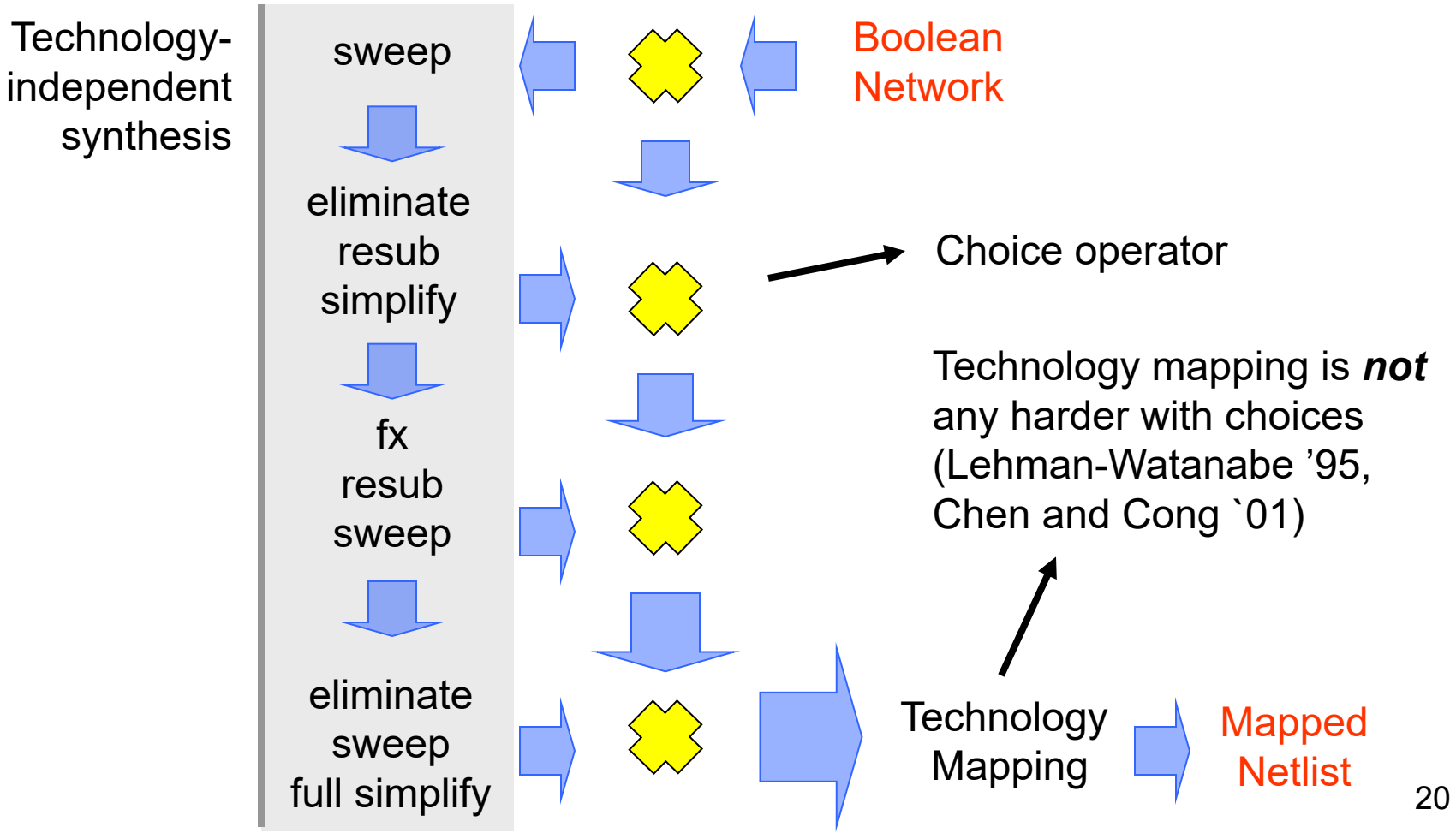
But structural bias means the mapped netlist depends heavily on the final network.

Technology Mapping

Mapped Netlist

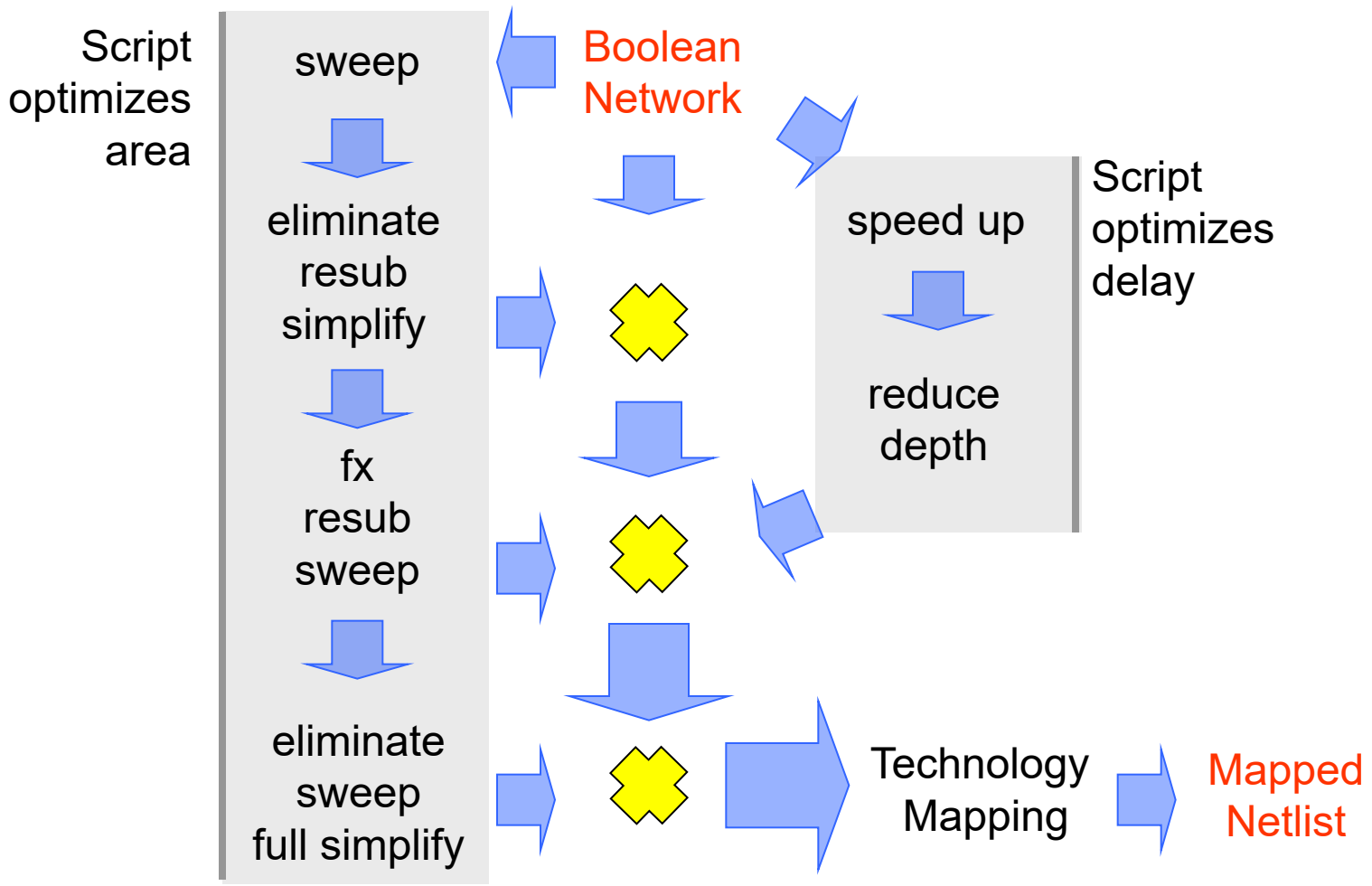
Lossless Synthesis

Idea: Merge intermediate networks into a single network with choices which is used for mapping

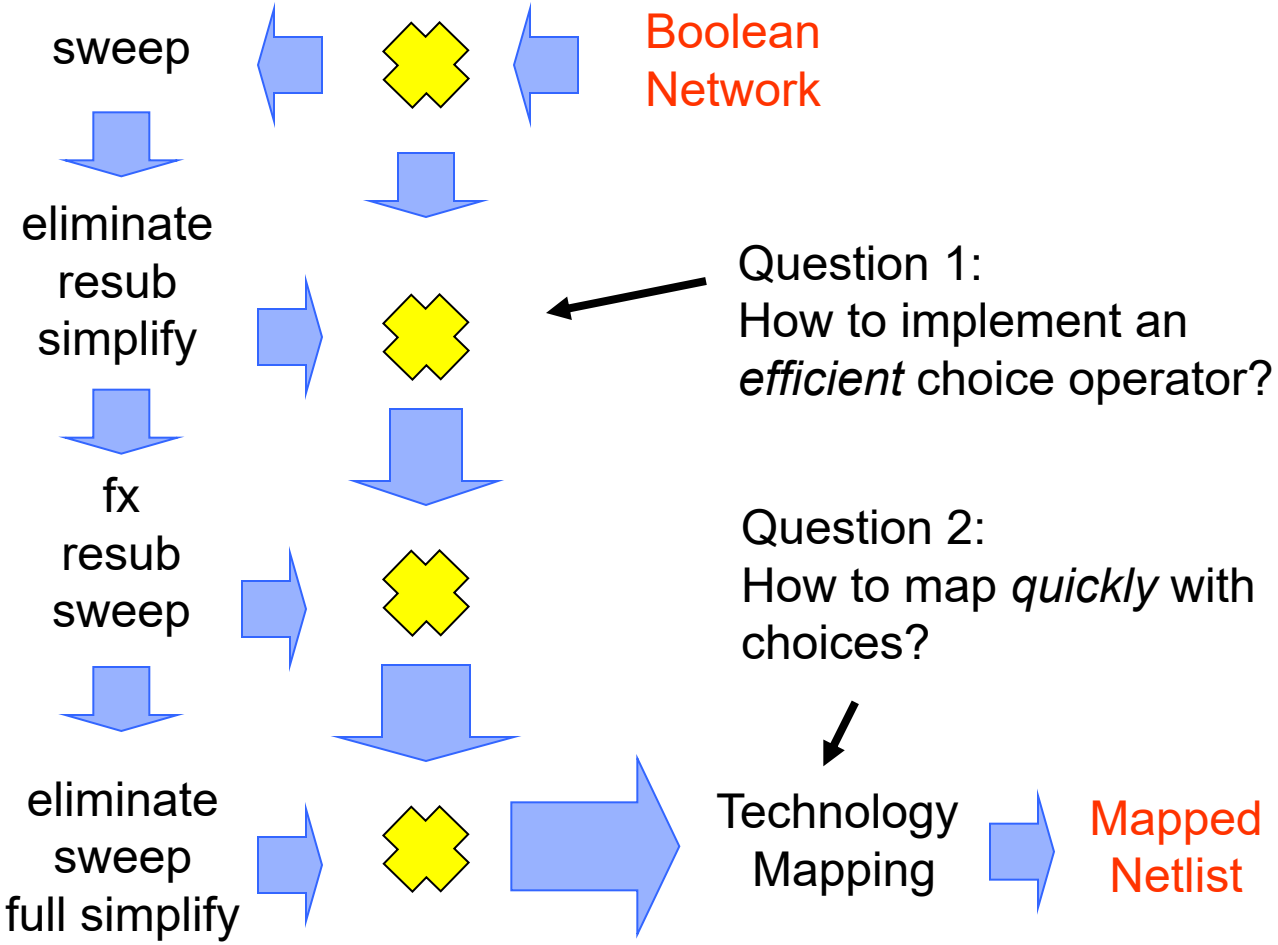


Lossless Synthesis

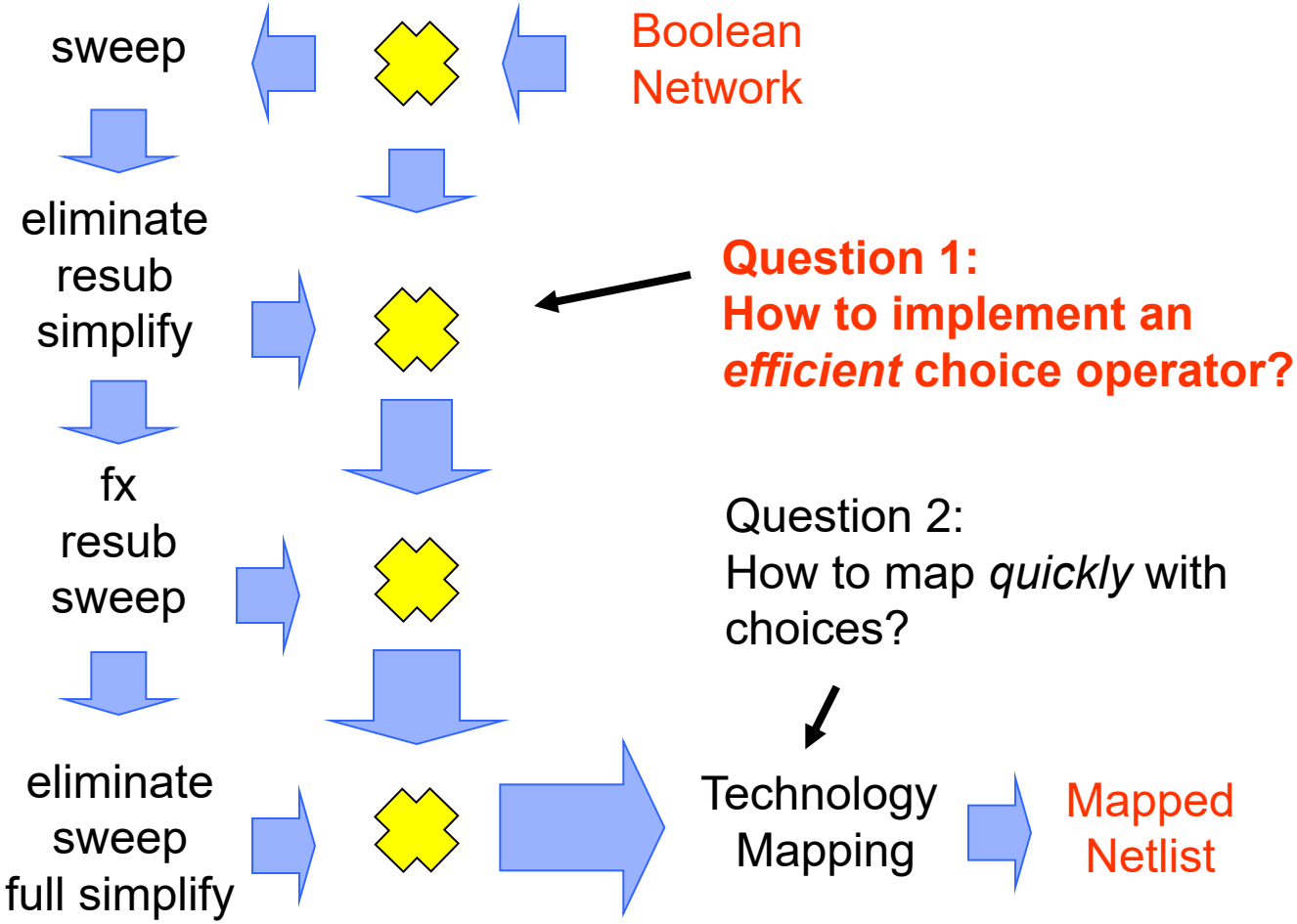
Can combine the results of different technology independent optimization scripts



Mapping with Choices



Mapping with Choices



Detecting Choices

Task: Given two Boolean networks, we need to create a network with choices

Network 1

x = (a + b).c

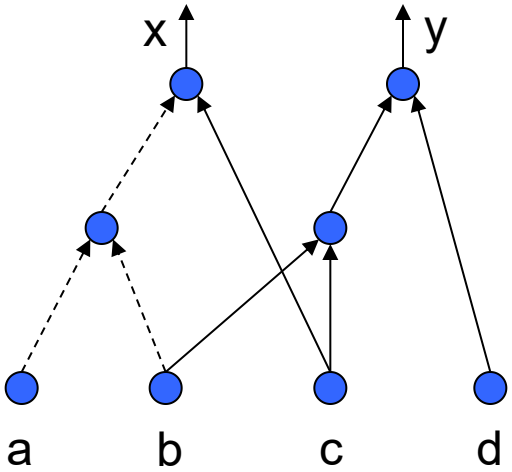
y = b.c.d

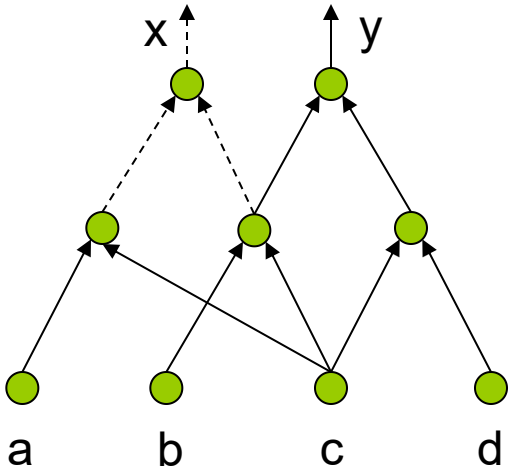
Network 2

x = a.c + b.c

y = b.c.d

Step 1: Make And-Inverter decomposition of networks





Detecting Choices

Step 2: Use combinational equivalence to detect functionally equivalent nodes up to complementation (Kuehlmann '04, …)

- Random simulation to detect possibly equivalent nodes
- SAT-based decision procedure to prove equivalence

Network 1

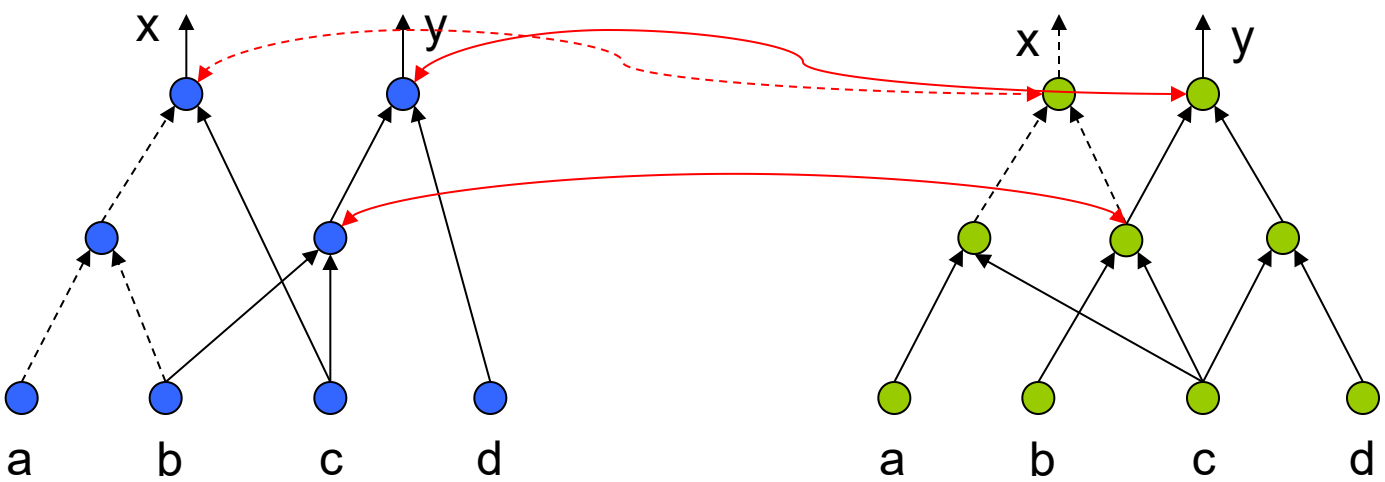
x = (a + b).c

y = b.c.d

Network 2

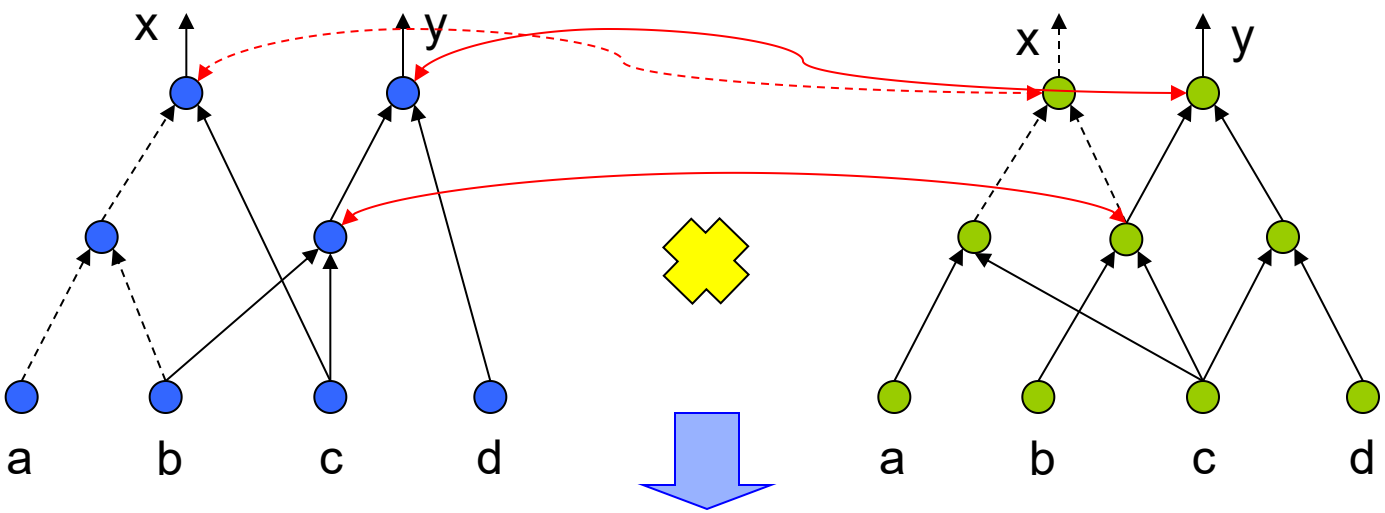
x = a.c + b.c

y = b.c.d

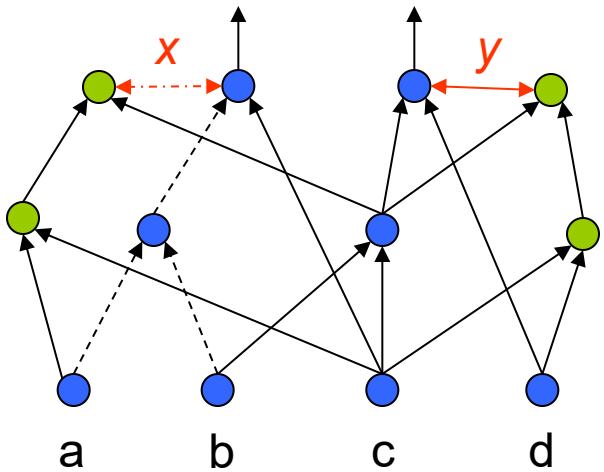


Detecting Choices

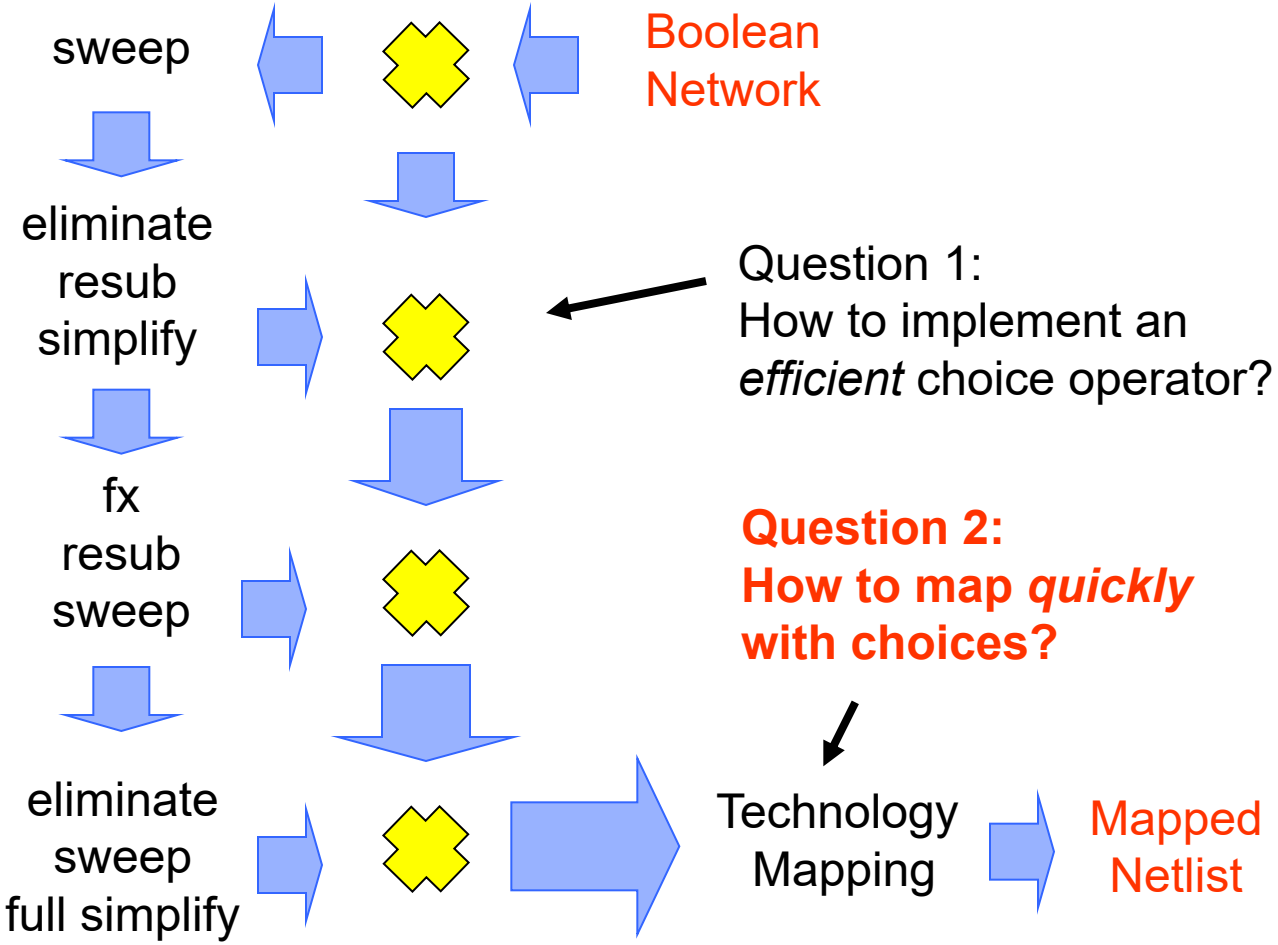
Step 3: Merge equivalent nodes with choice edges



x now represents a *class of nodes* that are functionally equivalent up to complementation



Mapping with Choices



Mapping with Choices

Only Step 1 requires modification

Input: And-Inverter Graph with Choices

1. Compute *k*-feasible cuts with choices
2. Compute best arrival time at each node
   - In topological order (from PI to PO)
   - Assuming that each cut maps to a *k*-LUT
   - Assuming that each *k*-LUT has unit delay
3. Chose the best cover
   - In reverse topological order (from PO to PI)

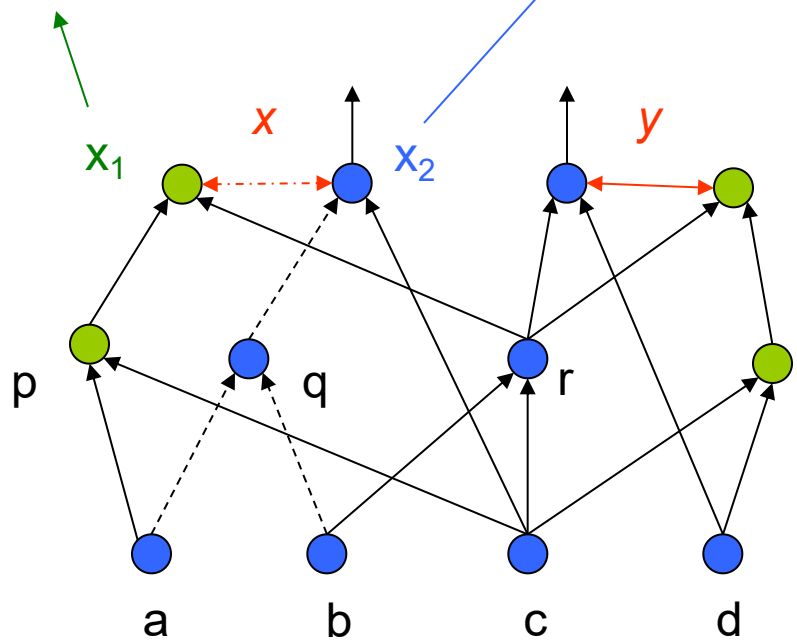
Output: Mapped Netlist

Cut Computation with Choices

Cuts are now computed for *equivalence classes* of nodes

{ {x_1}, {p, r}, {p, b, c}, {a, c, r}, {a, b, c} }

{ {x_2}, {q, c}, {a, b, c} }



Cuts (x) = Cuts (x_1) ∪ Cuts(x_2)

= { {x_1}, {p, r}, {p, b, c}, {a, c, r}, {a, b, c}, {x_2}, {q, c} }

Mapping with Choices

After Step 1 everything else remains same

Input: And-Inverter Graph with Choices

1. Compute *k*-feasible cuts with choices
2. Compute best arrival time at each node
   - In topological order (from PI to PO)
   - Assuming that each cut maps to a *k*-LUT
   - Assuming that each *k*-LUT has unit delay
3. Chose the best cover
   - In reverse topological order (from PO to PI)

Output: Mapped Netlist

Outline

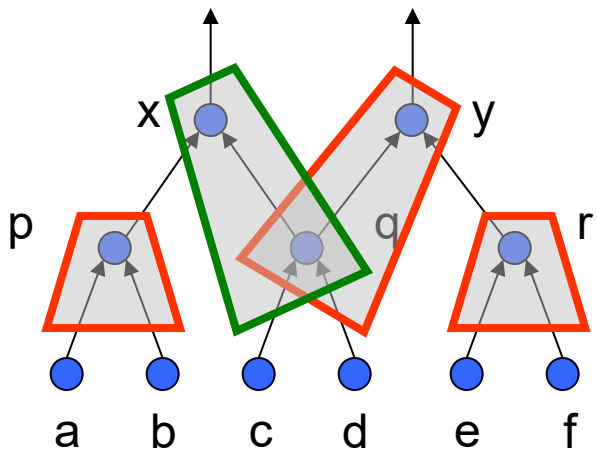
1. Review of Technology Mapping
2. More Efficient Cut Computation
3. Lossless Synthesis
4. Area Recovery
    1. Area-flow
    2. Exact Area

Overview of Area Recovery

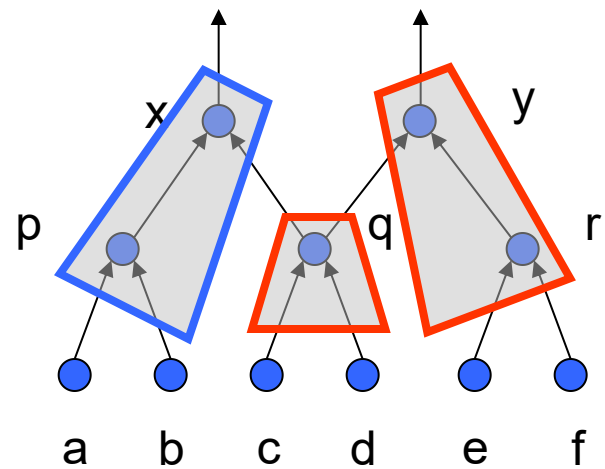
- Initial mapping is delay oriented
  - Gets best delay for all paths
  - Area-based tie-breaking
- Not all paths critical
  - Area recovery tries to slow down non critical paths to reduce area
  - Each node with positive slack: choose a different cut that reduces area
  - Done as subsequent passes after delay-oriented mapping
- Question: how to measure area?

How to Measure Area?

Naïve definition: Area (cut) = 1 + [Σ area (fan-in)]



Area of **cut {p, c, d}**
= 1 + [1 + 0 + 0]
= 2



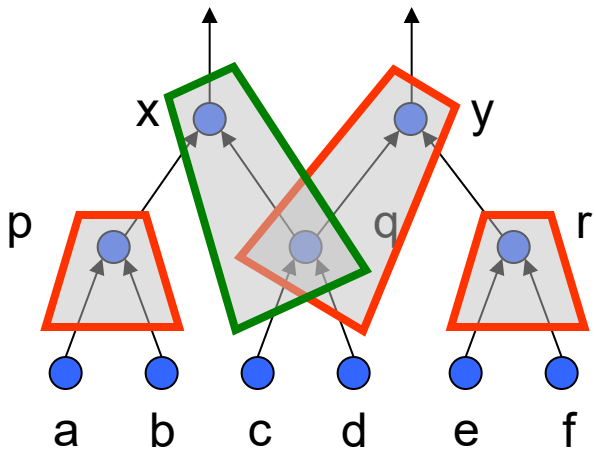
Area of **cut {a, b, q}**
= 1 + [0 + 0 + 1]
= 2

Naïve definition says both cuts are equally good in area

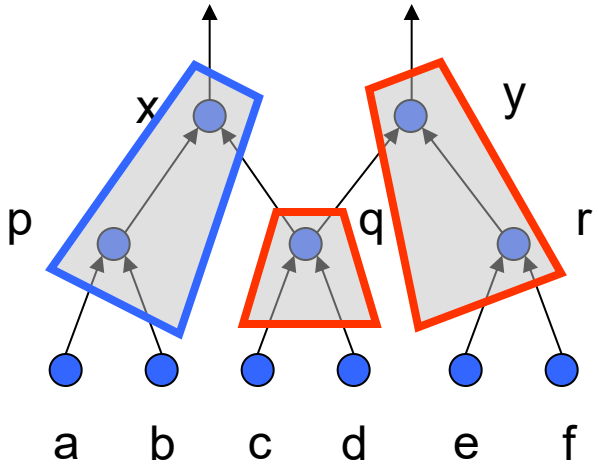
Naïve definition ignores sharing due to multiple fan-outs

Area-flow

Area-flow (cut) = 1 + [Σ (area-flow (fan-in) / fan-out (fan-in))]



Area-flow of **cut {p, c, d}**
= 1 + [1 + 0 + 0]
= 2



Area-flow of **cut {a, b, q}**
= 1 + [0/1 + 0/1 + ½]
= 1.5

Area-flow recognizes that **cut {a, b, q}** is better

Area-flow "correctly" accounts for sharing

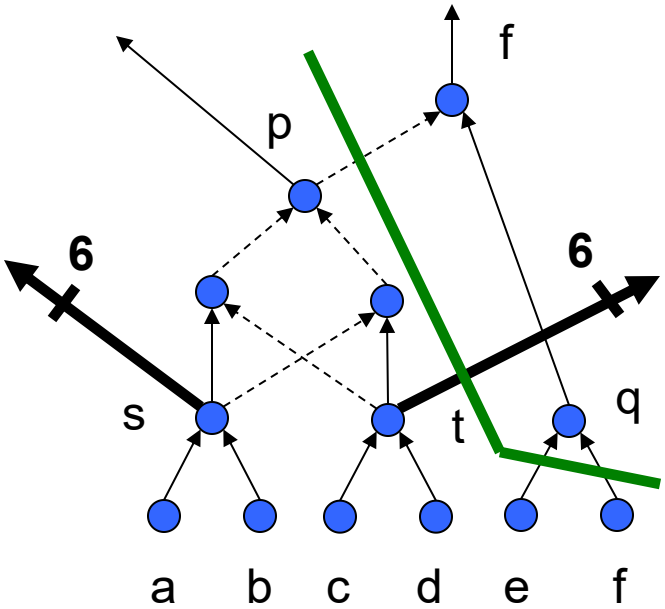
(Cong '99, Manohara-rajah '04)

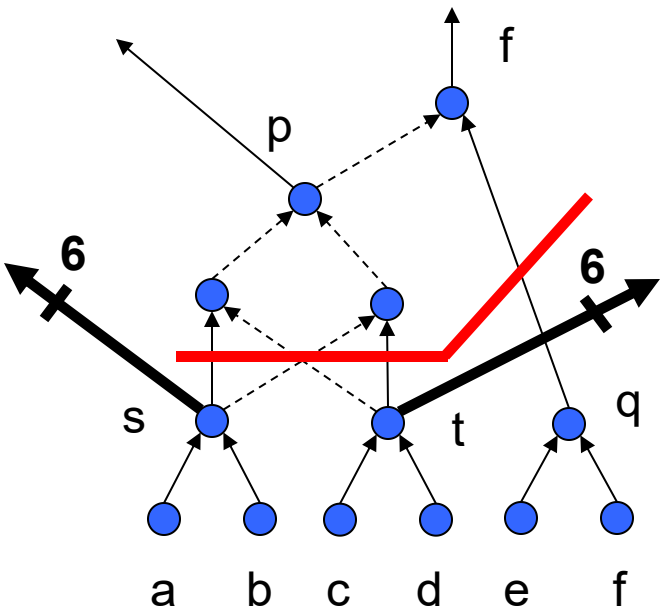
Area Recovery with Area-flow

1. Do delay-optimal mapping
2. Compute slack at each node
3. Do area recovery with area-flow
   - **Done in topological order from PI to PO**
   - Among all the cuts which do not exceed slack budget choose cut with smallest area-flow
   - Fan-out of a node is ***estimated*** from delay optimal mapping
   - **We only do one pass**
     - Saw only marginal improvement on subsequent passes

Exact Area

Exact-area (cut) = 1 + [Σ exact-area (fan-in with no other fan-out)]





Cut {p, e, f}

Area flow = 1+ [(.25+.25+3)/2] = 2.75

Exact area = 1 + 0 (p is used elsewhere)

Exact area will choose this cut.

Cut {s, t, q}

Area flow = 1+ [.25+.25 +1] = 2.5

Exact area = 1 + 1 = 2 (due to q)

Area flow will choose this cut.

Area Recovery with Exact-area

1. Do delay-optimal mapping
2. Compute slack at each node
3. Do area recovery with area-flow
4. Do area recovery with exact-flow
   - **Done in topological order from PI to PO**
   - Among all the cuts which do not exceed slack budget choose cut with smallest exact-area
   - Note: Unlike area-flow, **no estimation involved**
   - **We only do one pass**
     - Saw only marginal improvement on subsequent passes

Area Recovery Summary

- Two step area recovery
- Area-flow has global view
- Exact area has local view
  - Ensures local minimum is reached
- Order in which nodes are processed for both steps is important
- Order of the two passes is important

Experimental Comparison

- Compare area-recovery with state-of-the-art academic mapper DAOmap
  - DAOmap uses many (~10) different area recovery heuristics
  - Some more effective than others
- Just the two heuristics of area-recovery and exact-area give better results on their benchmarks
- Also separate comparison with choices obtained from lossless synthesis flow
  - Six snapshots of MVSIS script.rugged
  - Not the best FPGA optimization script ☺
  - Improves both area and delay

Comparison with DAOmap

Example	DAOmap			MVSIS-baseline			MVSIS-choices			MVSIS-choices 2x		
	Depth	LUTs	T, s	Depth	LUTs	T, s	Depth	LUTs	T, s	Depth	LUTs	T, s
alu4	6	1065	0.5	6	992	0.34	6	972	0.64	6	949	+0.84
apex2	7	1352	0.6	7	1200	0.36	7	1249	0.95	7	1191	+1.34
apex4	6	931	0.7	6	891	0.24	6	895	0.74	6	894	+1.47
bigkey	3	1245	0.6	3	797	0.34	3	797	0.75	3	684	+1.07
clma	13	5425	5.9	13	4426	1.50	11	3883	4.30	11	3453	+5.20
des	5	965	0.8	5	1024	0.36	5	947	0.93	5	1104	+1.87
diffeq	10	817	0.6	10	844	0.30	9	745	0.46	9	736	+0.43
dsip	3	686	0.5	3	686	0.23	3	685	0.19	3	684	+0.36
elliptic	12	1965	2.0	12	2017	0.61	12	2005	0.72	12	2022	+1.25
ex1010	7	3564	4.0	7	3258	1.15	7	3305	3.39	7	3302	+5.80
ex5p	6	778	1.0	6	744	0.36	5	724	1.17	5	675	+1.40
frisc	16	1999	1.9	15	2009	0.76	14	1875	1.54	13	1867	+1.58
misex3	6	980	0.8	6	957	0.26	6	926	0.73	6	861	+0.94
pdc	7	3222	4.6	8	2920	1.13	7	2738	4.73	7	2692	+5.59
s298	13	1258	2.4	13	826	0.30	12	863	4.07	11	826	+1.49
s38417	9	3815	3.8	9	3864	1.46	8	2989	4.04	7	2729	+2.76
s38584	7	2987	27.0	7	2844	1.11	7	2497	2.58	6	2470	+1.69
seq	6	1188	0.8	6	1109	0.30	5	1136	0.79	6	1016	+1.38
spla	7	2734	4.0	7	2535	1.03	7	2319	4.68	7	2224	+4.79
tseng	10	706	0.6	10	752	0.25	8	719	0.39	8	705	+0.31
Ratio	**1.00**	**1.00**	**1.00**	**1.00**	**0.93**	**0.37**	**0.95**	**0.89**	**0.95**	**0.93**	**0.86**	**1.46**

Summary

- Improvements to cut computation
  - Cut dropping
  - Signature-based dominance check
- Lossless Synthesis
  - Map over multiple synthesis snapshots
- Simpler, faster and better area recovery
  - Global area-flow
  - Local exact area
  - Order of application is important
- Implemented in the ***abc*** system
  - Google: “abc berkeley logic synthesis”